

WEEK 1 – LEARNING REPORT

Embedded System Basics

Name: Swayam Rudrawar

1. Objective

This week was focused on building a solid foundation for understanding embedded systems and microcontrollers before starting the Smart Monitoring System project. This meant learning how microcontrollers work, learning about GPIO pins and digital/analog signals, getting familiarized with STM32 and ESP boards, and setting up the development environments needed for the rest of the internship.

2. Topics Covered

2.1 Introduction to Microcontrollers and Embedded Hardware

I first began by learning what makes embedded systems different from general purpose computers. A microcontroller is a tiny computer that can fit on a single chip, and it contains a CPU, RAM, flash memory and a set of input/output pins all on the same chip, which means you can use it in small devices like sensors, home appliances and IoT boards instead of a full PC. A microcontroller normally runs one program, not many programs at once like a normal computer. Embedded code is normally written around a `setup()` and a continuous `loop()`.

I also explained the basic architecture of a microcontroller: the CPU core, the flash memory where the program is stored, SRAM for variables during runtime and the peripheral blocks like timers, ADC, communication interfaces. This helped me understand better what is actually happening inside the chip when I upload and run a program.

2.2 GPIO and Digital/Analog Signals

GPIO stands for General Purpose Input/Output. These are pins on the board that can be configured as input or output depending on what the program needs. A digital signal is only in two states, HIGH (around 3.3V on the ESP32) or LOW (0V). This is what we use to turn an LED on/off or read if a button is pressed. However, an analog signal can be any number in a continuous range of values, such as the voltage from a potentiometer or a sensor, which is read using the ADC (Analog to Digital Converter) built into the microcontroller.

One important thing I learned is that the ADC of the ESP32 gives a 12 bit value, that is, readings are from 0 to 4095, which is different from boards such as Arduino Uno that only give 0 to 1023 (10 bit). The mapping/scaling logic needs to be in line with the correct resolution and this is something that I had to keep in mind while writing the analog read program.

2.3 Basics of STM32/ESP Boards

Since the project will be using the ESP32, I focused mainly on understanding this board. The ESP32 has built-in Wi-Fi and Bluetooth, dual-core processing, multiple GPIO pins, and several ADC channels, which makes it well suited for an IoT-based smart monitoring system since it will eventually need to send sensor data to the cloud. I also briefly looked at the STM32 series to understand how it compares — STM32 boards are generally used where Wi-Fi/Bluetooth isn't needed and more focus is on precise control, lower power, or industrial-grade reliability.

2.4 Embedded Development Environments

I explored the common IDEs used for embedded development. The Arduino IDE is the simplest to get started with and is what I used for writing and uploading the three programs this week, since it supports the ESP32 directly through the board manager. I also installed PlatformIO (as a VS Code extension) and STM32CubeIDE to get a feel for more advanced/professional development setups, even though the actual programs for this internship were done using the Arduino IDE since it's the most practical for ESP32 + sensor work.

3. Tasks Performed

3.1 IDE Installation

Installed and set up the following development environments:

- Arduino IDE – installed ESP32 board package through Board Manager (esp32 by Espressif Systems)
- STM32CubeIDE – installed to get familiar with STM32 workflow
- PlatformIO – installed as a VS Code extension for an alternate, more advanced workflow
- Keil uVision – briefly explored for ARM-based microcontroller development

3.2 Basic Programs Written

Wrote and tested three basic embedded programs on the ESP32 (DevKit V1 board), listed below. Full code is attached as separate .ino files along with this report.

No.	Program	Description
1	LED Blink	Blinks the onboard LED (GPIO2) ON/OFF every 1 second, with status printed on Serial Monitor
2	Button Control	Reads a push button on GPIO4 (with internal pull-up) and turns an LED ON while the button is pressed
3	Analog Read (Potentiometer)	Reads analog voltage from a potentiometer on GPIO34 (ADC) and uses it to control LED brightness via PWM

3.3 Datasheet Study

Went through the ESP32 datasheet (ESP32-WROOM-32 module) to understand the pinout, operating voltage range (3.0V–3.6V), the number of usable GPIO pins, which pins are input-only (like GPIO34-39, used for the ADC pins in this internship), and which pins should be avoided for general use because they're connected internally to flash memory (GPIO6-11). This was useful while choosing pins for the LED, button, and potentiometer in the programs above, and will also help in deciding sensor pin placement for Week 2.

4. Observations / Learnings

- Got comfortable with the basic structure of embedded code (setup/loop) and how it differs from regular application programming.
- Understood the difference between digital and analog signals practically by comparing button input (digital) vs potentiometer input (analog).
- Learned that pin selection on ESP32 needs some care — not all GPIOs behave the same way (some are input-only, some are reserved internally).

- Realized the importance of checking the datasheet before wiring anything, instead of assuming all boards behave like Arduino Uno.
- Got the Arduino IDE + ESP32 setup fully working, which will be used for the rest of the project.

5.3 basic codes

Program 1 - LED Blink

Board: ESP32

Internship Week 1 - Embedded System Basics

Onboard LED on most ESP32 dev boards is connected to GPIO2.

If your board's LED is on a different pin, change LED_PIN below.

*/

```
#define LED_PIN 2
```

```
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
  Serial.println("LED Blink program started");
}
```

```
void loop() {
  digitalWrite(LED_PIN, HIGH);
  Serial.println("LED ON");
  delay(1000);

  digitalWrite(LED_PIN, LOW);
  Serial.println("LED OFF");
  delay(1000);
}
```

Program 2 - Button Control

Board: ESP32

Internship Week 1 - Embedded System Basics

Push button connected on GPIO4, one leg to GPIO4 and other leg to GND.

Using internal pull-up so we don't need an external resistor.

Button pressed = LOW (because of pull-up), not pressed = HIGH.

LED connected on GPIO2 (or use onboard LED).

*/

```
#define BUTTON_PIN 4
#define LED_PIN 2

int buttonState = 0;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
  Serial.println("Button control program started");
}

void loop() {
  buttonState = digitalRead(BUTTON_PIN);

  if (buttonState == LOW) {
    // button is pressed
    digitalWrite(LED_PIN, HIGH);
    Serial.println("Button pressed - LED ON");
  } else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("Button not pressed - LED OFF");
  }

  delay(200); // small delay to avoid flooding serial monitor
}
```

Program 3 - Analog Read (Potentiometer)

Board: ESP32

Internship Week 1 - Embedded System Basics

Potentiometer middle pin connected to GPIO34 (ADC1 channel).

Outer two pins of pot go to 3.3V and GND.

ESP32 ADC gives values from 0 to 4095 (12-bit), unlike Arduino Uno (0-1023).

This program reads the analog value and also maps it to a voltage and to a brightness percentage for the LED.

*/

```
#define POT_PIN 34
```

```
#define LED_PIN 2
```

```
void setup() {
```

```
  pinMode(LED_PIN, OUTPUT);
```

```
  Serial.begin(115200);
```

```
  Serial.println("Analog read program started");
```

```
}
```

```
void loop() {
```

```
  int rawValue = analogRead(POT_PIN);
```

```
  float voltage = (rawValue / 4095.0) * 3.3;
```

```
  int brightness = map(rawValue, 0, 4095, 0, 255);
```

```
  Serial.print("Raw ADC: ");
```

```
  Serial.print(rawValue);
```

```
  Serial.print(" Voltage: ");
```

```
  Serial.print(voltage);
```

```
  Serial.print("V Brightness: ");
```

```
Serial.println(brightness);

analogWrite(LED_PIN, brightness); // PWM controlled brightness

delay(500);
}
```